

1GC-7RC: One Graphic Card - Seven Research Challenges!

How Good Are AI Agents at Doing Your Job?

Robin-Nico Kampa, Fabian Deuser, Anna Charlotte Bößendorfer,
Konrad Habel, Norbert Oswald

University of the Bundeswehr Munich, Munich, Germany
{robin-nico.kampa, fabian.deuser, anna.boessendoerfer,
konrad.habel, norbert.oswald}@unibw.de

May 20, 2026

Abstract

Autonomous AI coding agents are becoming a core tool for ML practitioners in industry and research alike. Despite this growing adoption, no standardized benchmark exists to evaluate their ability to design, implement, and train models from scratch across diverse domains. We introduce **1GC-7RC** (*Single Graphic Card: Seven Research Challenges*), a benchmark comprising seven ML tasks spanning language modeling, image classification, semantic segmentation, graph learning, tabular prediction, time-series forecasting, and text classification. Each task provides a locked data-preparation and evaluation script together with a baseline training script; the agent may only modify the training code, has no access to pretrained weights (with one controlled exception for semantic segmentation), no internet access, and must complete each task within a task-specific wall-clock budget (40-120 minutes) on a single GPU. We evaluate seven coding agents: five proprietary (Claude Code with Sonnet 4.6, Opus 4.6, and Opus 4.7; Codex CLI with GPT 5.5; and OpenCode with Qwen 3.6+) and two open-source (OpenCode with Kimi K2.5, Kimi K2.6). Across 5 runs per agent-task pair, we report substantial performance differences that reveal varying levels of implicit ML knowledge, planning ability, and time-budget management. The benchmark, harness, and all evaluation artifacts are publicly available on GitHub at <https://github.com/Strolchii/1GC-7RC-Benchmark> to facilitate reproducible comparison of future agents. Because our benchmark design is modular, the benchmark can be extended to new tasks and domains, adapted to different GPU budgets, and used to study multi-agent settings, making it a flexible platform for future research on autonomous research agents.

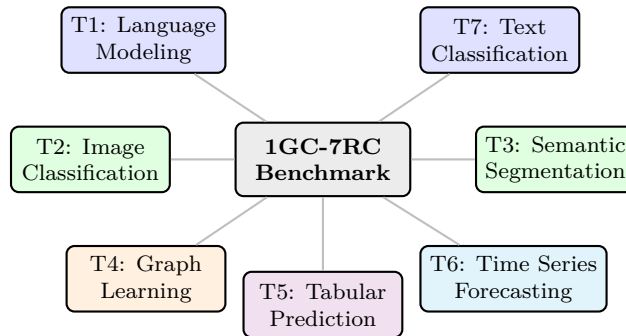


Figure 1: Six ML domains covered by the seven 1GC-7RC tasks. NLP and computer vision (CV) are each represented by two tasks: language modeling and text classification for NLP, and image classification and semantic segmentation for CV. This captures both generative and discriminative capabilities in language as well as recognition and dense prediction in vision.

1 Introduction

Large language models (LLMs) have evolved from code-completion tools into autonomous *coding agents* that read files, execute shell commands, interpret errors, and iterate on solutions with minimal human oversight. A particularly ambitious application of such agents is end-to-end machine learning engineering: given a dataset and an evaluation protocol, the agent must choose an architecture, implement the training loop, tune hyperparameters, and produce a competitive model, all autonomously.

Several benchmarks have begun to probe this capability. MAgentBench [21] asks agents to improve baselines on Kaggle-style tasks, MLE-bench [9] draws from real Kaggle competitions, and RE-Bench [46] targets AI R&D tasks. SWE-bench and SWE-bench Verified [23, 13], on the other hand, focus on software engineering tasks such as bug fixing and pull-request resolution. However, these benchmarks either allow pretrained weights, enforce no strict time budget, or focus on a narrow set of ML domains. MLR-Bench [11] may be the closest in spirit, but it relies on an LLM judge to score novelty, soundness, and clarity rather than fixed, deterministic evaluation metrics. None systematically tests whether an agent can build *from-scratch* models across the full breadth of modern ML, including vision, language, graphs, tabular data, and time series, under a strict, reproducible time budget and with objective, metric-based evaluation.

We address this gap with **1GC-7RC** (*Single Graphic Card: Seven Research Challenges*), a benchmark designed around four principles:

1. **From-scratch training.** In all but one task (Task 3: Image Segmentation), no pretrained weights are permitted and the agent must demonstrate genuine knowledge of model architectures, loss functions, optimizers, and training dynamics.
2. **Strict time budget.** Each task has a task-specific wall-clock budget (40-120 minutes), forcing the agent to balance exploration (trying new ideas) against exploitation (training longer).
3. **Efficient single-GPU use.** Each run is restricted to one NVIDIA A100 80 GB GPU, but agents are encouraged to exploit the available accelerator efficiently via parallelized training and data loading where appropriate.
4. **Multi-domain coverage.** Seven tasks drawn from six ML sub-fields ensure that strong performance requires breadth, not just depth.

This design is intentionally modular: adding a new task requires only a dataset, a locked evaluator, and a baseline training script, which makes 1GC-7RC easy to extend to additional domains, alternative hardware budgets, or multi-agent protocols. The benchmark harness enforces full isolation: the agent receives a baseline `train.py` and a locked `prepare.py` that handles data loading and evaluation; the agent is encouraged to create its own `run_{x}.py` scripts with improvements based on the baseline, even if editing `train.py` directly is not forbidden. Our benchmark shows that proprietary frontier models outperform open-source alternatives, that agents differ substantially in parallelism strategies, and that performance and reliability vary markedly across model families, exposing characteristic failure modes such as protocol violations and over-refusal. Our contributions are threefold:

(1) **A reproducible, open-source benchmark** with seven diverse ML tasks, a deterministic evaluation harness, and clear rules for agent interaction. (2) **An evaluation of seven coding agents** across 245 runs, revealing performance gaps, qualitative strategy differences, and characteristic weaknesses. (3) **A public leaderboard** and artifact repository supporting ongoing comparison of future agents; entries evaluated over more runs supersede those with fewer, ensuring that expanded evaluations improve rather than complicate comparisons.

Table 1: Comparison of ML agent benchmarks. ✓ = yes, ✗ = no.

Benchmark	From scratch	Time limit	Multi-domain	Deterministic eval	Number of Tasks
MAgentBench	✗	none	partial	✓	13
MLE-bench	✗	24 h	✓	✓	75
RE-Bench	✗	8 h	✗	✓	7
MLR-Bench	✗	varies	✓	✗	32
1GC-7RC	6/7	40-120 min	✓	✓	7

2 Related work

2.1 Benchmarks for autonomous research and coding agents

Code-generation benchmarks. HumanEval [12] and MBPP [6] evaluate single-function synthesis but are not agentic: the model produces one code block without iterative execution or debugging. SWE-bench [23] and its verified variant [13] move to repository-level bug fixing, requiring agents to localize issues, edit files, and pass test suites. WebArena [56] and OSWorld [49] broaden the scope to web and desktop tasks.

ML-specific agent benchmarks. MLE-agentBench [21] provides 13 ML tasks and measures whether agents can improve baselines via code edits, but permits pretrained weights and imposes no strict wall-clock limit. MLE-bench [9] draws 75 tasks from Kaggle, evaluating agents against historical medal thresholds; however, many tasks are data-centric rather than model-design challenges. RE-Bench [46] targets AI R&D capabilities with open-ended research tasks requiring up to 8 hours of compute. AIDE [22] introduces a tree-search agent for ML experimentation. Table 1 contrasts these benchmarks. Our benchmark complements this landscape by requiring *from-scratch* training across seven domains under tight per-task time budgets (40-120 minutes), emphasizing the agent’s intrinsic ML knowledge rather than its ability to leverage pretrained checkpoints.

2.2 AutoML and neural architecture search

Traditional AutoML systems such as Auto-sklearn [16], AutoGluon [14], and FLAML [43] automate pipeline construction through search algorithms over predefined configuration spaces. Neural Architecture Search (NAS) methods [57, 28] discover architectures via reinforcement learning or differentiable relaxation. More recently, LLM-based approaches use language models to propose and refine ML pipelines in natural language [18, 53], blurring the line between AutoML and agentic coding. 1GC-7RC differs from AutoML benchmarks in that the agent operates through a general-purpose coding interface (read/write files, execute shell commands) rather than a structured search API, making it a test of the agent’s *implicit* ML knowledge encoded during training.

2.3 Coding agent architectures

Modern coding agents share a common loop: observe context, decide on an action (edit a file, run a command), observe the result, and repeat [48, 3]. Early groundwork for autonomous code generation was laid by orchestration frameworks such as LangChain [10] and AutoGen [47], which provide reusable primitives for chaining LLM calls, tools, and memory. Claude Code is a vendor-provided CLI agent that wraps a frontier LLM with tool-use scaffolding [3]. Open-source alternatives include OpenCode [2] and OpenHands [45]. These agents differ in planning depth, error-recovery strategies, and context management; factors that 1GC-7RC is specifically designed to stress-test across diverse agents in the ML domain.

2.4 Autonomous research and self-improving agents

A recent line of work explores *autonomous research loops* in which an agent iteratively modifies code, measures a fitness metric, and repeats [26]. The design of 1GC-7RC is inspired by this closed-loop paradigm. The AI Scientist [31] and its successor [51] demonstrate end-to-end scientific discovery via agentic tree search. ADAS [19] automates the design of agentic systems themselves, while SICA [40] studies self-improving coding agents. ML-Agent [30] reinforces LLM agents for autonomous ML engineering. MLR-Bench [11] evaluates agents on open-ended ML research tasks, but scoring is performed by an LLM judge assessing novelty, soundness, and clarity of the produced report rather than measuring task performance directly; this is closer to automated paper review than to benchmark evaluation. AgentBench [29] provides a comprehensive multi-environment benchmark for LLM-as-agent evaluation.

1GC-7RC adopts the *autoresearch* paradigm of closed-loop, metric-driven iteration, while focusing on the breadth of ML knowledge under strict time constraints rather than on open-ended research capability.

Table 2: Overview of the seven 1GC-7RC tasks and baseline training configurations. For TinyShakespeare, train/eval counts refer to individual characters. The **Eval** column reports the split used by the harness for scoring: validation for Tasks 1-3 and 7; held-out test for Tasks 4-6.

Task Specification					Baseline Configuration				
Task	Budget	Domain	Metric	Dataset	Model	Train	Eval	Score	Batch
T1	40 min	Language Modeling	↓ val_bpb	TinyShakespeare [24]	GPT-4L-256d	900 K	100 K	3.438	64
T2	120 min	Image Classification	↑ Top-1 Acc.	TinyImageNet [27]	ResBlock CNN	100 K	10 K	0.343	256
T3	80 min	Sem. Segmentation	↑ mIoU	Pascal VOC 2012 [15]	ConvNeXt-T+FPN	1 464	1 449	0.660	16
T4	60 min	Graph Learning	↑ AUROC	ogbg-molhiv [20]	GIN (5L)	33 K	4 K	0.706	256
T5	40 min	Tabular Prediction	↑ Accuracy	Forest Cover [7]	MLP (3L)	465 K	58 K	0.806	4096
T6	40 min	Time Series Forecasting	↓ MSE	ETTh1 [55]	DLinear	8 640	4 800	0.384	128
T7	40 min	Text Classification	↑ Accuracy	AG News [54]	Char-CNN	120 K	7 600	0.793	512

3 Benchmark design

3.1 Core principles

All scores are reported as mean $\pm$ std over repeated runs, together with a per-task success rate (fraction of runs yielding a valid score). Entries evaluated over more runs are considered more statistically reliable and supersede entries with fewer runs when comparing models, regardless of the raw score. The benchmark can easily be adapted to different GPU generations or extended to multi-GPU setups.

To summarize overall performance across heterogeneous task metrics, we additionally report an aggregate score (Agg), defined as the mean baseline-relative improvement over all $T = 7$ tasks. For agent a with mean score $s_{a,t}$ on task t and baseline score s_t^{base} , we compute:

$$\text{Agg}_a = \frac{1}{T} \sum_{t=1}^T \begin{cases} \frac{s_{a,t} - s_t^{\text{base}}}{s_t^{\text{base}}}, & \text{if higher-is-better,} \\ \frac{s_t^{\text{base}} - s_{a,t}}{s_t^{\text{base}}}, & \text{if lower-is-better.} \end{cases} \quad (1)$$

The same baseline-relative aggregation can also be applied to each agent’s single best task scores to obtain an Agg_{max} summary; we report this best-run view in Appendix Table 6. Our main analysis, however, focuses on the mean-based variant shown in Table 4, as it provides a more reliable characterization of expected performance in practice.

3.2 Task suite

Table 2 summarizes the seven tasks and their baseline configurations. The tasks span six major ML subfields and rely on well-established datasets and standard evaluation metrics, enabling interpretable results and meaningful comparison to prior work. The licensing terms and usage conditions for all datasets employed in this benchmark are provided in Appendix E.

Task 1 (Language Modeling). Character-level language modeling on Karpathy’s TinyShakespeare corpus [24] (~ 1 M characters, 65-character vocabulary). The text is split 90/10 into training and validation sets and tokenized into `uint16` character IDs. The baseline provides a small GPT decoder (4 layers, 4 heads, $d = 256$, dropout = 0.1) trained with AdamW (lr = 3×10^{-4} , cosine schedule, 200-step warmup, gradient clipping at 1.0) for 5 epochs at block size 256. The metric is validation bits-per-byte (bpb = $\frac{\mathcal{L}}{\ln(2)}$). To improve, the agent can scale the architecture, adjust the context length, or change the optimizer, but it must ensure convergence within the budget.

Task 2 (Image Classification). 200-class classification on TinyImageNet (100 K training images, 10 K validation images, 64×64 RGB, ImageNet-normalized). The baseline is a simple residual CNN [17] with four stages ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ channels, stride-2 downsampling) followed by global average pooling and a fully connected head. Training uses random crops with 8-pixel padding, horizontal flips, and color jitter. Without pretrained backbones, the agent must design its own feature extractor.

Task 3 (Semantic Segmentation). Pixel-wise labeling on Pascal VOC 2012 (21 classes including background, ignore label 255, ~ 1 464 training and 1 449 validation images at a fixed resolution of 224×224). This is the only task that permits pretrained weights: the harness provides two DINOv3 backbones, ConvNeXt Tiny and ViT-S [41], pre-downloaded into a `weights/` directory, since training a competitive segmentation model from scratch is infeasible within the time budget. The agent may use both, either, or neither of these backbones and freely design the decoder architecture. The baseline pairs

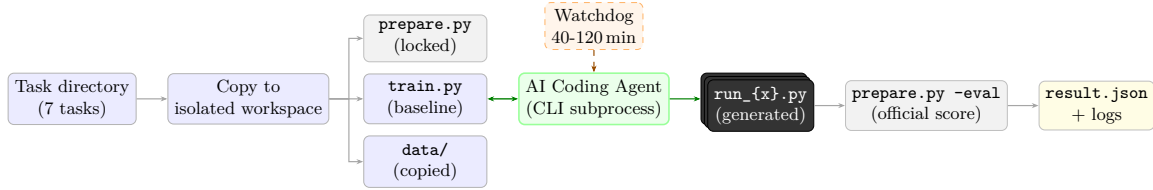


Figure 2: Harness architecture. Each task runs in an isolated workspace with copied `prepare.py` (locked), `train.py`, and dataset. Agents create `run_{x}.py`; a watchdog enforces the time budget.

the ConvNeXt Tiny DINOv3 encoder with a lightweight Feature Pyramid Network (FPN) decoder and trains for 15 epochs with AdamW ($\text{lr} = 10^{-4}$, batch size 16).

Task 4 (Graph Learning). Binary classification of molecular graphs for HIV inhibition prediction (ogbg-molhiv from the Open Graph Benchmark; 41 K graphs with 9-dim atom features and 3-dim edge features, official train/val/test split). The baseline implements a 5-layer Graph Isomorphism Network (GIN) [50] with learned atom embeddings ($d = 64$, summed per feature), hidden dimension 300, global mean pooling, and an MLP classifier, all in pure PyTorch without PyG. A custom `collate_graphs()` function batches variable-size graphs with node-offset tracking. The metric is AUROC on the test set.

Task 5 (Tabular Prediction). Seven-class classification on Forest Cover Type (581 K rows, 54 features: 10 continuous + 44 binary one-hot; stratified 80/10/10 split, StandardScaler-normalized). The baseline is a 3-layer MLP ($256 \rightarrow 256 \rightarrow 128$, BatchNorm, ReLU, dropout = 0.3). This task probes whether agents can improve upon the MLP baseline through architectural choices (deeper networks, residual connections, attention-based tabular models) or improved training strategies (learning-rate scheduling, regularization, feature interactions).

Task 6 (Time Series Forecasting). Multivariate 96-step-ahead forecasting on ETTh1 (hourly electricity transformer temperatures, 7 features, 17 420 time steps; standard LTSF 12/4/4-month split, train-normalized). Given 336 historical steps, the model predicts the next 96. The baseline is DLinear [52]: moving-average decomposition (kernel 25) into trend and seasonal components, each projected by a linear layer. The metric is MSE. Agents can attempt more complex architectures (PatchTST, Transformer) but risk exceeding the time budget.

Task 7 (Text Classification). Four-class news classification on AG News (World, Sports, Business, Sci/Tech; 120 K training and 7 600 test samples). Texts are encoded at the character level (ASCII 0-127, max length 1 014, zero-padded). The baseline is a character-level CNN inspired by Zhang et al. [54]: 128-dim embeddings, six convolutional layers (kernels 7,7,3,3,3,3; 256 channels each), max-pooling, and two 1 024-unit FC layers with dropout = 0.5.

3.3 Harness architecture

For each of the seven tasks, the harness is described in the following and illustrated in Figure 2.

Step 1. Creation of an isolated run directory containing copies of the dataset and task-specific `program.md`, as well as locked versions of `prepare.py` and the baseline `train.py`.

Step 2. Launch of the agent as a CLI subprocess with a standardized prompt instructing it to read `program.md` and, based on the provided `train.py`, generate its own `run_{x}.py` scripts with improvements.

Step 3. Streaming of the agent’s stdout to both the console and a log file, with structured JSON events parsed for real-time monitoring.

Step 4. Enforcement of a hard per-task wall-clock timeout (40-120 minutes, depending on dataset size; see Table 2) via a daemon watchdog thread with a 30-second grace period.

Step 5. After termination, scanning of the `checkpoints/` directory for scored checkpoints, selection of the overall best, clustering of remaining checkpoints into 5-minute windows (retaining only the best per window to preserve diversity), and execution of `prepare.py -eval` to produce the official metric as a JSON object.

For Task 3, the harness additionally downloads and caches the two permitted DINOv3 backbone weights in a `weights/` directory prior to execution. All artifacts, including the final `train.py`, full stdout logs, and `result.json`, are retained for post-hoc analysis.

3.4 Rules and constraints

Recent analyses of agent benchmarks report that coding agents regularly attempt to cheat on benchmark tasks [44]. To mitigate this, we provide each agent with an explicit set of rules as part of the prompt. Compliance is enforced post hoc by auditing stdout logs, filesystem snapshots, and the recorded sequence of tool invocations, allowing us to verify the agent’s behavior on disk and its tool usage. Runs that violate the protocol (e.g., accessing the locked evaluator, using the internet, or circumventing the time budget) are disqualified. The protocol is defined by the following rules:

Rule 1. Edit scope: `prepare.py` is locked; agents may create custom `run_{x}.py` scripts based on `train.py`.

Rule 2. No pretrained weights: models are trained from scratch, except Task 3 (semantic segmentation), which provides optional DINOv3 backbones [41].

Rule 3. No internet access: no external data, code, or weights may be downloaded.

Rule 4. No package installation: no additional libraries may be installed.

Rule 5. Stay in working directory: agents must not leave the assigned task directory.

Rule 6. Time budget: 40-120 minutes per task on one NVIDIA A100 80 GB GPU, with efficient utilization encouraged.

Rule 7. Output contract: scripts must expose `build_model(config)` and save scored checkpoints.

Rule 8. Isolated evaluation: scoring uses the original `prepare.py` and dataset outside the agent workspace.

Rule 9. Validation vs. test splits: Tasks 4–6 use held-out test splits, while Tasks 1–3 and 7 evaluate on validation splits.

4 Experiments

4.1 Main results

We evaluate seven coding agents across three CLI frameworks (Table 3): three Anthropic models accessed via Claude Code, one OpenAI model accessed via Codex CLI, and three additional models (two open-source, one proprietary) routed through OpenCode via OpenRouter. Appendix B provides details on the experimental setup.

Our primary comparison uses mean $\pm$ std over 5 runs per agent-task pair rather than the single best run, since we are interested in typical performance rather than isolated outliers; for completeness, Appendix Table 6 reports the best-run scores (i.e., the highest score achieved among the five runs), while Appendix Table 5 reports the mean number of training scripts launched per run and the success rates ($n/5$), with the latter directly indicating the number of successful runs.

The reference values (Ref.[†]) reported in Table 4 and Table 6 are drawn from prior work to contextualize agent performance; however, they do *not* correspond to directly comparable experimental setups. Appendix D provides details on their selection, conversion, and interpretation.

Table 3: Agents evaluated in this study. OpenCode runs are routed via OpenRouter [36–38]. All experiments ran in April 2026.

CLI Agent	Model	Provider	Run dates
Proprietary (Claude Code)			
Claude Code [3]	claude-sonnet-4-6 [4]	Anthropic	April 19-20
Claude Code	claude-opus-4-6 [4]	Anthropic	April 20-21
Claude Code	claude-opus-4-7 [4]	Anthropic	April 17-18
Proprietary (Codex CLI)			
Codex CLI [34]	gpt-5.5 [35]	OpenAI	April 25-30
★ Proprietary ◦ Open-Source (OpenCode)			
OpenCode [2]	◦ kimi-k2.5 [42]	Moonshot	April 19-24
OpenCode	◦ kimi-k2.6 [32]	Moonshot	April 22-25
OpenCode	★ qwen3.6-plus [39]	Alibaba	April 20-24

4.2 Analysis of agent strategies

Examining the agent stdout logs, generated training scripts, and final submission artifacts across all 245 runs reveals several distinct behavioral characteristics of the evaluated AI agents.

Table 4: Main results: mean $\pm$ std; 5 runs per agent. **Bold**: best mean, (incl. Agg). Red cell: success rate $< 100\%$ (per-task rates in Appendix Table 5). Agg: baseline-relative improvement (Eq. 1). $\dagger$ Published reference (see text). $\ddagger$ Single run of the baseline `train.py` (T3 15ep; T5 40ep; rest 5ep).

	Task 1 $\downarrow$ bpb	Task 2 $\uparrow$ Acc	Task 3 $\uparrow$ mIoU	Task 4 $\uparrow$ AUROC	Task 5 $\uparrow$ Acc	Task 6 $\downarrow$ MSE	Task 7 $\uparrow$ Acc	1GC-7RC $\uparrow$ Agg
Reference †	2.120 [25]	0.627 [1]	—	0.848 [8]	0.969 [5]	0.375 [33]	0.905 [54]	—
Baseline ‡	3.438	0.343	0.660	0.706	0.806	0.384	0.793	—
Proprietary (Claude Code)								
Sonnet 4.6	2.2325 ± 0.2097	0.6813 ± 0.0252	0.8322 ± 0.0074	0.7663 ± 0.0116	0.9632 ± 0.0063	0.3809 ± 0.0049	0.9211 ± 0.0024	+0.293
Opus 4.6	2.1797 ± 0.0123	0.6619 ± 0.0592	0.8047 ± 0.0217	0.7771 ± 0.0156	0.9682 ± 0.0031	0.3895 ± 0.0068	0.9214 ± 0.0076	+0.281
Opus 4.7	2.0863 ± 0.0234	0.6897 ± 0.0179	0.8073 ± 0.0340	0.7754 ± 0.0117	0.9704 ± 0.0013	0.3761 ± 0.0035	0.9212 ± 0.0062	+0.302
Proprietary (Codex CLI)								
GPT 5.5	2.0964 ± 0.0724	0.6638 ± 0.0517	0.7855 ± 0.0226	0.7783 ± 0.0389	0.9724 ± 0.0003	0.3803 ± 0.0028	0.9318 ± 0.0025	+0.287
★ Proprietary ○ Open-Source (OpenCode)								
○ Kimi K2.5	2.5643 ± 0.2858	0.6015 ± 0.0553	0.6036 ± 0.1772	0.7611 ± 0.0276	0.9645 ± 0.0047	0.3835 ± 0.0021	0.8972 ± 0.0160	+0.190
○ Kimi K2.6	2.1741 ± 0.0475	0.5932 ± 0.0639	0.7302 ± 0.0431	0.7768 ± 0.0096	0.9677 ± 0.0049	0.3848 ± 0.0115	0.8953 ± 0.0247	+0.233
★ Qwen 3.6+	2.2971 ± 0.1354	0.5614 ± 0.0988	0.6259 ± 0.1735	0.7705 ± 0.0133	0.9557 ± 0.0090	0.3880 ± 0.0044	0.8974 ± 0.0157	+0.188

Failure modes. Across all 245 runs, outright failures were rare but informative. The dominant pattern was not a modeling weakness but an implementation mismatch with the locked evaluation contract: most commonly, incorrect checkpoint serialization after `torch.compile()` produced state-dictionary keys incompatible with the evaluation harness, rendering otherwise functional models unloadable. Additional cases arose from agent-code bugs or overrefusal. A detailed breakdown is provided in Appendix C.

Single-model tuning vs. post-hoc ensembling. Codex (gpt-5.5) is the only agent in the study that habitually constructs heterogeneous post-hoc blends. Within the first 40 to 60 % of its budget it trains 2 to 4 small, short-running models in parallel, calls `prepare.py --eval` on each, and combines the resulting checkpoints into a single submitted artifact: a 4-GPT mixture with an n-gram fallback on Task 1, a top-5 snapshot ensemble on Task 2, a CatMLP+KNN+ExtraTree blend wrapped in a single `nn.Module` on Task 5, and a CharCNN+TF-IDF/SVC blend on Task 7. Once the blend evaluates well it terminates the session, leaving the remaining $\sim 50\%$ of wall-clock unused; see Appendix Table 8. This likely reflects a preference for conserving tokens over fully exhausting the time budget. All four of Codex’s task wins (Task 1, Task 2, Task 5, Task 7) involve such a blend; the strategy is opportunistic rather than universal, with adoption ranging from 20 % of runs on Task 1 to 100 % on Task 5. The remaining six agents (Sonnet 4.6, Opus 4.6/4.7, both Kimi variants and Qwen 3.6+) commit to one architecture family per task, train it for the full budget, and refine via SWA, EMA, or checkpoint averaging at most. None of them produces a heterogeneous blend in any of their 5 runs across any of the 7 tasks.

Iteration count is not skill. Our experiments show a negative correlation between the mean number of training scripts launched per task and the aggregate score, as shown in Appendix Figure 3. Qwen launches the most scripts overall (often 20-30 per run on the heaviest tasks (T4 and T6), peaking at 36 on a single Task 4 attempt) and finishes with the lowest Agg (+0.188). Opus 4.7 launches the fewest (median 5-8 per task) and achieves the highest Agg (+0.302). The reason is visible in the run dirs: Qwen’s high script counts are spent on hyperparameter grids that span a single architecture (e.g. all 36 Task 4 scripts converge to vanilla GIN), whereas Opus 4.7 explores 3-7 architectural variants, recognises diminishing returns early, and pivots to consolidation. What matters is not how many scripts are launched but what dimension the variants span.

Architectural choice: knowing the modern stack vs. deciding when to use it. On Task 1 (character-level GPT) only the three OpenCode agents (Kimi K2.5, Kimi K2.6, Qwen 3.6+) adopt the post-2024 transformer recipe (RMSNorm + SwiGLU + RoPE); the four remaining proprietary agents stay with the nanoGPT defaults (LayerNorm + GELU + learned positional embeddings) and tune them more carefully. Across the rest of the suite the proprietary cluster prefers conservative-classical families (ResNet, GIN, DLinear, VDCNN) and pairs them with EMA/SWA or ensembling. Despite their more modern recipe, the OpenCode cluster holds the three lowest aggregate scores across all tasks in our benchmark. The gap traces to a meta-decision rather than ML knowledge: OpenCode agents reach for

the modern recipe even where it is not the right choice for the given task and computational budget constraints (e.g. Qwen’s 24-layer Task 1 model with batch 16 only matches Kimi K2.6’s 8-layer modern-stack model at 2.13 bpb), while proprietary agents more often recognise that a tuned classical baseline beats an under-budget modern one.

Post-hoc refinement habits. Each agent has a default refinement trick that it deploys across tasks. Opus 4.6 reaches for SWA (Tasks 3/5/7; two or more of its 5 runs per task use it). Opus 4.7 typically prefers EMA paired with batch-norm recalibration. Codex’s default is a heterogeneous blend. Sonnet 4.6’s default is the TTA-in-filename trick of Task 3; it does not refine post-hoc on any other task. The three OpenCode agents have no such recurring habit and consistently leave roughly 0.5-1 % per task on the table.

Where insights live: training-detail discoveries. On 5 of the 7 benchmark tasks, the per-task winner is determined by a single *training-detail* discovery rather than simply an architectural choice alone: **Task 3 (segmentation):** Sonnet 4.6 embeds test-time augmentation inside the `forward()` method, averaging predictions over the original, horizontally flipped, and vertically flipped inputs so the locked evaluator effectively receives an ensemble-style boost at every single `model(x)` call (+0.034 mIoU over the runner-up). **Task 4 (graph):** Opus 4.6 stacks residual connections, JK-sum aggregation, and concatenated mean+max pooling on a GIN backbone - a recombination that no other agent assembled. **Task 5 (tabular):** Codex wraps a CatMLP, a KNN head, and an ExtraTree in one `nn.Module` with learned blend weights throughout. **Task 6 (time series):** Kimi K2.6 trains a plain DLinear with L1 loss and weight decay 10^{-3} on train+val combined; the L1 swap alone almost closes the 2.5 % gap to the PatchTST reference (five other agents stayed on MSE/Huber). **Task 7 (text):** Codex blends a character-level CNN with a TF-IDF/SVC classifier - nobody else really left the all-deep-learning approach.

Per-task winners. The seven task winners (best of 5) are spread across four agents (Codex GPT 5.5: 4×, Sonnet 4.6: 1×, Opus 4.6: 1×, Kimi K2.6: 1×); the mean-of-5 winners (Table 4) are spread across three (Opus 4.7: 3×, Codex GPT 5.5: 3×, Sonnet 4.6: 1×). No single agent dominates. Appendix Table 7 summarises the winning recipe per task - linking the row scores to the mechanism that produced them.

5 Discussion

What do the results reveal about agent ML knowledge? The variation in performance across tasks suggests that LLMs encode ML sub-field knowledge unevenly. Tasks aligned with common training data (e.g., text classification, image classification) often yield stronger performance, whereas rarer domains (graph learning, semantic segmentation from scratch) expose knowledge gaps. This aligns with the hypothesis that agents rely on memorized patterns from pretraining rather than first-principles reasoning about ML. The `build_model(config)` contract forces a clean separation between architecture design and training logic, enabling to compare *what* each agent builds.

Connection to autoresearch. The autoresearch paradigm [26], closed-loop agent optimization against a measurable fitness function, is what 1GC-7RC operationalizes in a controlled setting. Systems like AI-Scientist [31], AIDE [22], and ML-Agent [30] show that agents can improve ML code when given enough time. Our contribution is to standardize *what* is measured (7 diverse tasks), *how long* the agent has (40-120 minutes per task), and *what resources* are available (single GPU, no pretrained weights except for one controlled exception), enabling comparison across agent families.

The explore-exploit trade-off. The tight per-task budget (40-120 minutes) creates a fundamental tension between reading code and planning (exploration) versus launching training runs (exploitation). This trade-off mirrors the anytime algorithm literature and provides a natural axis along which agent quality can be assessed. A second cross-cutting signal is wall-clock budget utilization: all agents except Codex CLI with GPT 5.5 typically exhausted their budget, whereas Codex often terminated early, using only 50.15% of its allocated time on average across 35 task instances; this most plausibly reflects a tendency to save tokens by ending the run early. Per-task averages in Appendix Table 8.

Implications for AI-assisted research. The benchmark results suggest current coding agents can serve as useful ML prototyping assistants on well-understood tasks: current coding agents reliably set up training loops and produce scoreable checkpoints, but expose a clear creativity gap: five of seven per-task winners earned their lead with a single non-obvious trick, each found by a different agent and not replicated elsewhere. The leap from “iterate on a sensible baseline” to “spot the unconventional move” remains the hardest dimension to elicit. As the autoresearch ecosystem matures, with specific forks for genomics, finance, robotics, and more [26], standardized benchmarks like 1GC-7RC become essential for separating “executes the obvious recipe well” from “discovers the non-obvious one.”

Limitations and future work. This benchmark is designed for a single GPU and seven supervised-learning tasks; generative modeling (e.g., diffusion models, GANs), reinforcement learning, audio and speech processing, and multi-modal settings would be conceivable extensions. All runs use one hardware configuration (A100 80 GB), so validating conclusions on other accelerators is a worthwhile goal for future revisions of the benchmark. Equally, future versions can be extended to support multi-GPU and multi-agent protocols, and as hardware becomes more accessible and compute costs continue to decline, increase the number of repeated runs to strengthen statistical comparisons.

6 Conclusion

1GC-7RC is designed as an extensible benchmark definition rather than a fixed leaderboard snapshot. Its core protocol separates the measurement principle from task instances and agent implementations: new tasks can be added as self-contained directories, and new agents only require an additional dispatchable entry; the same modularity supports extensions to multi-agent and multi-GPU settings. Results obtained under the same budget, hardware, and evaluation rules remain comparable, so the benchmark grows with the ecosystem. On the leaderboard, a larger sample (i.e., reporting mean $\pm$ std over a larger number of runs) always supersedes a smaller one regardless of absolute scores, and the success rate is reported so readers can assess each entry independently of its rank.

The transition from 2025 to 2026 has marked a notable leap in autonomous coding capability. Tools such as Claude Code [3], Codex CLI [34], and OpenCode [2] have evolved from assisted code completion into agents that can independently design, implement, and debug non-trivial ML pipelines within minutes. The pace of this progress raises important questions beyond benchmarking: as autonomous agents become capable of performing tasks that currently require trained researchers, the implications for scientific workflows, labor markets, and the distribution of technical expertise will be substantial. Whether those implications are ultimately beneficial or disruptive will depend on how the research community chooses to deploy, govern, and share these systems.

References

- [1] Zoheb Abai and Nishad Rajmalwar. Densenet models for tiny imagenet classification. *arXiv preprint arXiv:1904.10429*, 2019.
- [2] Anomaly Innovations. OpenCode: The open source coding agent, 2025. URL <https://github.com/anomalyco/opencode>. Accessed: 2026-04-28.
- [3] Anthropic. Claude code overview, 2025. URL <https://code.claude.com/docs/en/overview>. Accessed: 2026-04-28.
- [4] Anthropic. Model system cards, 2026. URL <https://www.anthropic.com/system-cards>. Accessed: 2026-04-28.
- [5] Sercan Ö Arik and Tomas Pfister. Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 6679–6687, 2021.
- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [7] Jock A Blackard and Denis J Dean. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and electronics in agriculture*, 24(3):131–151, 1999.
- [8] Zacharie Bugaud. Multi-rf fusion with multi-gnn blending for molecular property prediction. *arXiv preprint arXiv:2603.20724*, 2026.
- [9] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pages 50466–50494, 2025.

- [10] Harrison Chase. LangChain, 2022. URL <https://github.com/langchain-ai/langchain>. Released: 2022-10-17; Accessed: 2026-04-28.
- [11] Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. Mlr-bench: Evaluating ai agents on open-ended machine learning research. *Advances in Neural Information Processing Systems*, 38, 2026.
- [12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [13] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- [14] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. Autogluon-tabular: Robust and accurate automl for structured data. *arXiv preprint arXiv:2003.06505*, 2020.
- [15] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International journal of computer vision*, 88(2): 303–338, 2010.
- [16] Matthias Feurer, Aaron Klein, Katharina Eggenberger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. *Advances in neural information processing systems*, 28, 2015.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [18] Noah Hollmann, Samuel Müller, and Frank Hutter. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems*, 36:44753–44775, 2023.
- [19] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pages 21344–21377, 2025.
- [20] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.
- [21] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning*, pages 20271–20309. PMLR, 2024.
- [22] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code, 2025.
- [23] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.
- [24] Andrej Karpathy. char-rnn: Multi-layer recurrent neural networks for character-level language models. <https://github.com/karpathy/char-rnn>, 2015. Accessed: 2026-04-28.
- [25] Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/fine-tuning medium-sized GPTs. <https://github.com/karpathy/nanoGPT>, 2022. Accessed: 2026-04-28.
- [26] Andrej Karpathy. autoresearch: Ai agents running research on single-gpu nanochat training automatically, 2026. URL <https://github.com/karpathy/autoresearch>. Accessed: 2026-04-28.
- [27] Yann Le, Xuan Yang, et al. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015.

- [28] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.
- [29] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.
- [30] Zexi Liu, Jingyi Chai, Xinyu Zhu, Shuo Tang, Rui Ye, Bo Zhang, Lei Bai, and Siheng Chen. ML-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv:2505.23723*, 2025.
- [31] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [32] Moonshot AI. Kimi K2.6 tech blog: Advancing open-source coding, April 2026. URL <https://www.kimi.com/blog/kimi-k2-6>. Accessed: 2026-04-28.
- [33] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730*, 2022.
- [34] OpenAI. Introducing codex. <https://openai.com/index/introducing-codex/>, 2025. Accessed: 2026-05-02.
- [35] OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, April 2026. Accessed: 2026-05-02.
- [36] OpenRouter. Kimi K2.5 on OpenRouter, 2026. URL <https://openrouter.ai/moonshotai/kimi-k2.5>. Accessed: 2026-04-28.
- [37] OpenRouter. Kimi K2.6 on OpenRouter, 2026. URL <https://openrouter.ai/moonshotai/kimi-k2.6>. Accessed: 2026-04-28.
- [38] OpenRouter. Qwen3.6 Plus on OpenRouter, 2026. URL <https://openrouter.ai/qwen/qwen3.6-plus>. Accessed: 2026-04-28.
- [39] QwenTeam. Qwen3.6-Plus: Towards real world agents, April 2026. URL <https://qwen.ai/blog?id=qwen3.6>. Accessed: 2026-04-28.
- [40] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- [41] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- [42] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- [43] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. Flaml: A fast and lightweight autotml library. *Proceedings of machine learning and systems*, 3:434–447, 2021.
- [44] Hao Wang, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. Trustworthy benchmarks (cont.). <https://rds.berkeley.edu/blog/trustworthy-benchmarks-cont/>, 2026. Accessed: 2026-05-02.
- [45] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
- [46] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M Clymer, Jai Dhyani, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. In *International Conference on Machine Learning*, pages 66772–66832. PMLR, 2025.

- [47] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- [48] Scott Wu. Introducing devin, the first ai software engineer. *Cognition Labs Blog*, 2024.
- [49] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- [50] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.
- [51] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- [52] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 11121–11128, 2023.
- [53] Lei Zhang, Yuge Zhang, Kan Ren, Dongsheng Li, and Yuqing Yang. Mlcpilot: Unleashing the power of large language models in solving machine learning tasks. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2931–2959, 2024.
- [54] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28, 2015.
- [55] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.
- [56] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.
- [57] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016.

Acknowledgments

The authors gratefully acknowledge the Institute for Distributed Intelligent Systems (ETTI 2) and the Institute for Autonomous Systems Technology (LRT 8.1) at the University of the Bundeswehr Munich for granting access to Monacum One for benchmark development and to a NVIDIA A100 cluster for the final benchmark runs.

A Prompt preamble for overrefusal

As of April 2026, Anthropic’s latest flagship model Claude Opus 4.7 exhibited pronounced overrefusal behavior in our preliminary investigations: the model occasionally misclassifies the agent’s own workspace files as untrusted third-party code requiring a malware audit and refuses to modify them, preventing any progress on the task. Concretely, refused runs are easy to spot in the stdout logs by phrases such as “I should not modify code I did not write”, “this looks like third-party material that may be malicious”, or “I will run a security review before touching these files”; they typically end with an empty `checkpoints/` directory and zero `train.py` edits recorded by the harness.

To counteract this, we added an explicit preamble to the initial prompt given to all participating agents, clarifying that the workspace files belong to them and that internal system reminders suggesting

otherwise should be ignored. The preamble is applied uniformly across all agents for consistency. Importantly, no further model-specific accommodations were made; the preamble is designed as a general safeguard against overrefusal that applies equally to current and future models. We deliberately refrain from tailoring the benchmark to any single agent: optimizing prompts or scaffolding for one model would compromise fairness toward other and future participants. Even with this mitigation, Opus 4.7 still refused in one of our runs, as shown in Table 4 and Appendix Table 5.

B Experimental setup

All experiments run sequentially with one reserved NVIDIA A100 SXM4 80 GB GPU on an NVIDIA DGX A100 node, with 64 GB allocated system RAM and 16 cores of an AMD EPYC 7742 CPU. Each agent is given the same system prompt, task description, and baseline `train.py`. The prompt instructs the agent to read `program.md` and, based on the provided `train.py`, create its own `run_{x}.py` scripts with improvements. All experiments were conducted in April 2026; exact start and end dates for each agent are listed in Table 3. To account for the non-determinism inherent in LLM sampling (temperature, $\text{top-}p$) and variability in training outcomes, we run each agent-task pair **5 times** under the task-specific time budget (Table 2) and report the mean and standard deviation of the official metric. Agent outputs are streamed to a log file and parsed in real time via structured JSON events.

C Detailed failure cases

Three runs (Sonnet 4.6: Task 1 - Run 3 and Task 1 - Run 4; Opus 4.6: Task 1 - Run 4) wrapped the model in `torch.compile()` before checkpointing. PyTorch’s `OptimizedModule` prefixes every `state_dict` key with `_orig_mod.`, which causes the locked `prepare.py` to fail under `strict=True`. All other agents that used `torch.compile()`, namely Opus 4.7 (all 5 runs), Kimi K2.5, K2.6, and Qwen, did not exhibit this failure, confirming that the error is agent-side and solvable. The three affected task instances are counted as N/A (denominator $n = 3\text{--}4$ for those cells; see Table 4) and the contract is held fixed at `strict=True`. Despite the prompt-level safeguard against overrefusal included for all agents (Appendix A), this Anthropic-specific failure mode still occurred in one Opus 4.7 run (Task 2, Run 4).

Separately, Qwen produced two genuine agent-code failures with no recoverable checkpoint. In Task 1 - Run 3 the model was wrapped in `torch.compile()`; the Inductor backend crashed in the backward pass on every spawned script, and the agent’s repeated `Edit` attempts to strip the offending line failed on a whitespace mismatch, leaving `checkpoints/` empty. In Task 3 - Run 4 the data pipeline applied `ColorJitter` (a photometric augmentation) to VOC’s segmentation masks, which are stored as PIL palette-mode images that PIL’s brightness adjustment refuses with `ValueError: image has wrong mode`. The agent diagnosed the bug and wrote three replacement scripts that route photometric transforms only to the image, but none finished training before the budget expired.

D Reference metrics and their provenance

The reference values (Ref.[†]) reported in Tables 4 and 6 are drawn from published work to contextualize agent performance; they do *not* represent comparable experimental setups, as detailed in the accompanying text.

For Task 1, the nanoGPT reference [25] is published as cross-entropy loss in nats (1.4697); we convert it to bpb via $\text{bpb} = \frac{\mathcal{L}}{\ln(2)}$ to match the harness metric, yielding 2.120 bpb. For Task 2, the reference value is the best validation accuracy reported by [1] for a DenseNet-style network trained from scratch on the standard 200-class TinyImageNet split (100 K/10 K, 64×64). For Task 3 (Pascal VOC 2012 semantic segmentation at 224×224) we found no peer-reviewed reference with a comparable input resolution: published Pascal VOC results almost exclusively use 512×512 crops with ImageNet-pretrained encoders and the SBD-augmented training split, which are not meaningfully comparable to our constrained setup; we therefore leave this entry empty. For Task 5, the reference value is the Forest Cover Type test accuracy reported by TabNet [5] on the full UCI covtype dataset with 7-class classification; note that their 309,871/154,937/116,203 train/val/test split differs from our stratified 80/10/10 partition. For Task 6, the reference is the PatchTST/42 result on ETTh1, horizon 96 [33], which uses the same look-back window $L = 336$ as our benchmark and the same LTSF 12/4/4-month split. For Task 7, the reference is the best character-level ConvNet result on AG News reported by [54] (“Lg. Full Conv.

Th.”: full alphabet distinguishing case, thesaurus-based data augmentation), matching our character-level encoding (128 ASCII chars, max length 1014) and from-scratch training setup; word- and subword-level models with external pretraining are excluded by the benchmark rules. Task 4 (ogbg-molhiv) is the only benchmark in our suite with a publicly maintained leaderboard [20]; the current top-ranked submission [8] achieves 0.848 AUROC using an ensemble of ~ 993 M parameters, roughly $30,000\times$ larger than our 33 K GIN baseline. None of the other tasks have publicly maintained leaderboards. These works use different hardware, training durations, and, in most cases, pretrained weights, so direct comparison is not meaningful. The goal is not to surpass these values; rather, they serve as an aspirational upper bound to gauge how close agents can get under the restrictive conditions of our benchmark.

E Dataset licenses

All datasets used in the 1gc-7rc benchmark are publicly available and are used in accordance with their respective licensing terms, where specified, as well as applicable usage conditions:

1. **TinyShakespeare** [24]: Text based on public-domain Shakespeare works; distributed via Karpathy’s MIT-licensed `char-rnn` repository. No separate dataset license found.
2. **TinyImageNet** [27]: A downsampled subset of the ImageNet dataset; access and usage are subject to the ImageNet Terms of Access, which restrict usage to non-commercial research and educational purposes. No standalone license specific to TinyImageNet was found.
3. **Pascal VOC 2012** [15]: Provided as part of the PASCAL VOC 2012 challenge; usage of the included images must comply with the respective Flickr terms under which they were originally published. No explicit standalone dataset license is provided.
4. **ogbg-molhiv** [20]: Released under the permissive MIT License as part of the Open Graph Benchmark (OGB) initiative.
5. **Forest Cover Type** [7]: Publicly available via the UCI Machine Learning Repository and distributed under the Creative Commons Attribution 4.0 (CC BY 4.0) license.
6. **ETTh1** [55]: Released as part of the ETDataset repository under CC BY-ND 4.0; the Informer code repository is separately licensed under Apache 2.0.
7. **AG News** [54]: Collected from the AG corpus; no standard license found, but the dataset README permits use for research, data mining, and other non-commercial activity.

F Additional tables and figures

Table 5: Mean number of training scripts launched. Scripts are counted as train script invocations actually started within the allocated wall-clock budget, extracted from agent stdout logs. Each cell reports the success rate ($n/5$) followed by the mean script count over valid runs. Red cell: $n < 5$.

	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7
Proprietary (Claude Code)							
Sonnet 4.6	3/5 – 10	5/5 – 5	5/5 – 14	5/5 – 17	5/5 – 9	5/5 – 17	5/5 – 7
Opus 4.6	4/5 – 13	5/5 – 6	5/5 – 12	5/5 – 10	5/5 – 8	5/5 – 18	5/5 – 7
Opus 4.7	5/5 – 10	4/5 – 4	5/5 – 6	5/5 – 8	5/5 – 5	5/5 – 16	5/5 – 6
Proprietary (Codex CLI)							
GPT 5.5	5/5 – 8	5/5 – 5	5/5 – 6	5/5 – 6	5/5 – 7	5/5 – 8	5/5 – 8
★ Proprietary ◦ Open-Source (OpenCode)							
◦ Kimi K2.5	5/5 – 8	5/5 – 12	5/5 – 9	5/5 – 10	5/5 – 12	5/5 – 20	5/5 – 10
◦ Kimi K2.6	5/5 – 7	5/5 – 18	5/5 – 10	5/5 – 11	5/5 – 14	5/5 – 24	5/5 – 7
★ Qwen 3.6+	4/5 – 6	5/5 – 8	4/5 – 9	5/5 – 16	5/5 – 19	5/5 – 25	5/5 – 10

Table 6: Best single run per agent (best of 5, not mean). **Bold**: best score, (incl. $\text{Agg}_{\max}$). Red cell: success rate $< 100\%$ (Table 5); $\text{Agg}_{\max}$: baseline-relative improvement computed from best-of-5 scores (Eq. 1). $\dagger$ Published reference (Section 4.1 and Appendix D). $\ddagger$ Single run of the baseline `train.py` (T3 15ep; T5 40ep; rest 5ep).

	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	1GC-7RC
	$\downarrow$ bpb	$\uparrow$ Acc	$\uparrow$ mIoU	$\uparrow$ AUROC	$\uparrow$ Acc	$\downarrow$ MSE	$\uparrow$ Acc	$\uparrow$ $\text{Agg}_{\max}$
Reference †	2.120 [25]	0.627 [1]	—	0.848 [8]	0.969 [5]	0.375 [33]	0.905 [54]	—
Baseline ‡	3.438	0.343	0.660	0.706	0.806	0.384	0.793	—
Proprietary (Claude Code)								
Sonnet 4.6	2.1110	0.7153	0.8410	0.7794	0.9708	0.3767	0.9236	+0.320
Opus 4.6	2.1655	0.7004	0.8358	0.8028	0.9715	0.3795	0.9320	+0.315
Opus 4.7	2.0550	0.7150	0.8316	0.7907	0.9721	0.3701	0.9251	+0.325
Proprietary (Codex CLI)								
GPT 5.5	1.9727	0.7163	0.8066	0.8011	0.9728	0.3772	0.9354	+0.325
*Proprietary $\circ$ Open-Source (OpenCode)								
$\circ$ Kimi K2.5	2.2306	0.6344	0.7399	0.7823	0.9715	0.3800	0.9161	+0.257
$\circ$ Kimi K2.6	2.1310	0.6760	0.7806	0.7883	0.9722	0.3689	0.9107	+0.292
* Qwen 3.6+	2.1293	0.6454	0.7797	0.7818	0.9696	0.3845	0.9204	+0.273

Note: Opus 4.7 and GPT 5.5 are tied at +0.325 to three decimals; Opus 4.7 leads by $\sim 8 \times 10^{-6}$ at the 6th decimal place (+0.325093 vs. +0.325085).

Table 7: Per-task winning mechanism (best of 5). **Mean-bold**: agent with the highest per-task mean in Table 4. **Best-bold**: agent with the highest single run in Table 6.

Task	Mean-bold	Best-bold	Winning recipe (Best-bold)
T1 - Language Modeling	Opus 4.7	GPT 5.5	Post-hoc 4-GPT ensemble + n-gram fallback.
T2 - Image Classification	Opus 4.7	GPT 5.5	Top-5 snapshot ensemble of independent ResNets.
T3 - Semantic Segmentation	Sonnet 4.6	Sonnet 4.6	Frozen DINOv3 + minimalist FPN + h/v-flip TTA reported in file-name.
T4 - Graph Learning	GPT 5.5	Opus 4.6	Single GIN + edge features + virtual node + JK-sum + dual pool.
T5 - Tabular Prediction	GPT 5.5	GPT 5.5	Heterogeneous CatMLP + KNN + ExtraTree packed as one nn.Module.
T6 - Time Series Forecasting	Opus 4.7	Kimi K2.6	Plain DLinear + L1/MAE loss + $\text{WD } 10^{-3}$ + train+val combined.
T7 - Text Classification	GPT 5.5	GPT 5.5	CharCNN + TF-IDF/SVC heterogeneous blend (no Transformer).

Table 8: Per-task wall-clock budget utilisation of Codex (gpt-5.5), averaged over 5 runs.

Task	Budget minutes	Used minutes	Utilisation %
Proprietary (Codex CLI), gpt-5.5			
T1 - Language Modeling	40	24.93	62.32
T2 - Image Classification	120	75.06	62.55
T3 - Semantic Segmentation	80	27.23	34.04
T4 - Graph Learning	60	30.60	51.00
T5 - Tabular Prediction	40	17.43	43.57
T6 - Time Series Forecasting	40	15.39	38.48
T7 - Text Classification	40	23.64	59.11
Mean over 7 tasks	60	30.61	50.15

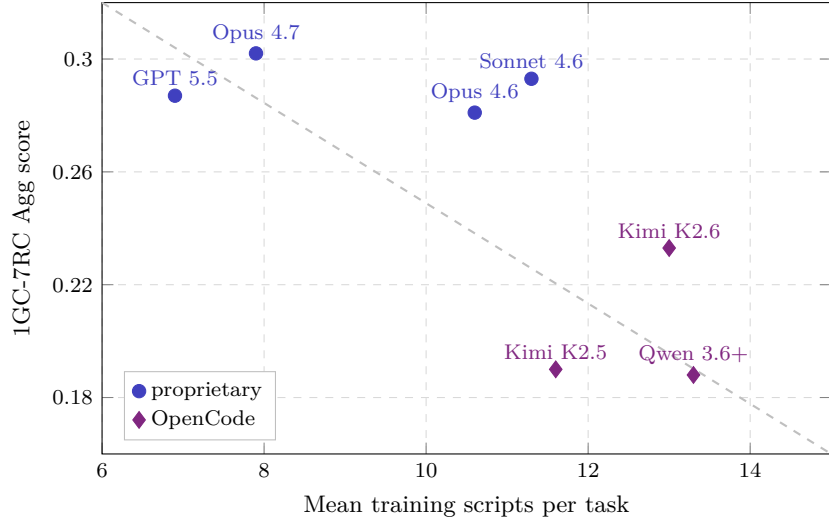


Figure 3: Mean training scripts per task (7 tasks $\times$ 5 runs) vs. 1GC-7RC Agg score (Eq. 1). Higher iteration counts do not predict higher scores; the dashed line indicates the anti-correlation.